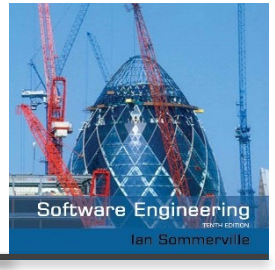
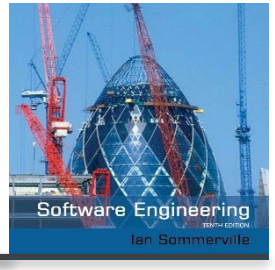


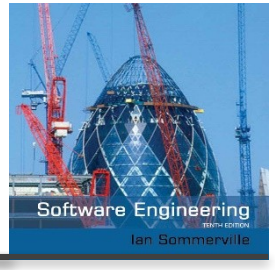
Chapter 4 – Requirements Engineering

Requirements engineering



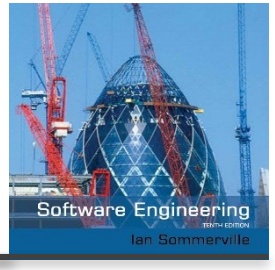
- **Requirements:**
  - the s**ervices that a customer requires** from a system
  - & the **constraints under which it operates**
- **Requirement engineering:**
  - The process of finding out, analysing, documenting and checking the constraints on system.
- Requirements may be used for :
  - Creating a bid for the contract
  - Creating a contract
- **Two types:**
  - User requirements
  - System requirements

Types of requirement



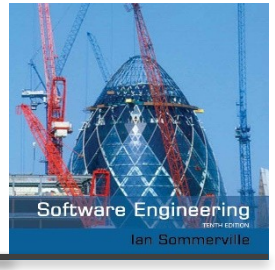
- **User requirements**
  - **Written in natural language, non technical terms**
  - Show the **high level view of the services** the system provides and its **operational constraints**.
  - **Written for stakeholders/managers/non technical end users**.
- **System requirements**
  - **A structured document**
  - Mention the **detailed descriptions** of the system's functions, services and operational constraints.
  - Defines **what should be implemented** so may be part of a **contract between client and contractor.**

stakeholders



- Any person / organization who has a legitimate interest in system and is affected directly by system progress
- types
  - End users
  - System managers
  - System owners
  - External stakeholders (regulatory bodies who passed the acceptance tests)

User and system requirements for a Hospital Management System



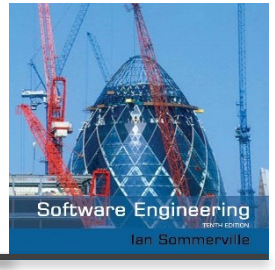
User requirements

- A user can make an appointment

System requirements

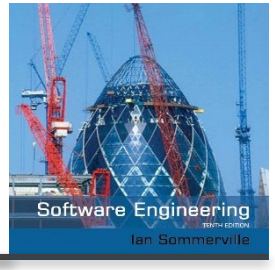
- User must have an account (sign up)
- User will login to that account
- Check for the specific doctor’s availability
- Make an appointment
- Pay the fee

Stakeholders for a Hospital Management System



- **Patients** whose information is recorded in the system.
- **Doctors** who are responsible for assessing and treating patients.
- **Nurses** who coordinate the consultations with doctors and administer some treatments.
- **Medical receptionists** who manage patients' appointments.
- **IT staff** who are responsible for installing and maintaining the system.
- **Health care managers** who use statistics derived from the system for hospital improvement.

Agile’s take on requirements engineering



- In agile, detailed system requirements are waste of time.
- Concurrent requirement elicitation
- No requirement document
- express requirements as ‘user stories’.

Functional and non-functional requirements

Functional and non-functional requirements

Functional Requirements

- **List the services provided by the system.**
- **System's response to a particular inputs** / adverse scenario.
- May state **what the system should or should not do.**
- Depend on:
  - type of software being developed
  - Expected users

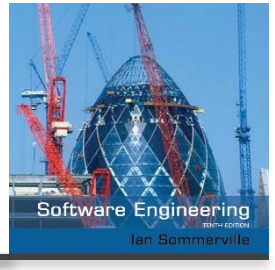
Non-functional Requirements

- **Constraints on the services offered by the system**
- Includes **timing constraints**, **constraints on the development process**, **standards** etc.
- Often **apply to the system as a whole rather than individual services.**
- May relate to system properties like **reliability, memory usage, response time, security, usability etc.**
- NF requirements may be more critical than functional requirements. **If these are not met, the system might be useless.**
- **Example**:
  - **a banking application**, to ensure security, may give the authorization to user only after authentication.
  - To **reduce response time in an embedded system**, minimum hardware components are used to perform

Domain Requirements

- Special constraints on system enforced in case of a specific domain application
  - Example:
    - System availability or Network BW in case of a live streaming website.

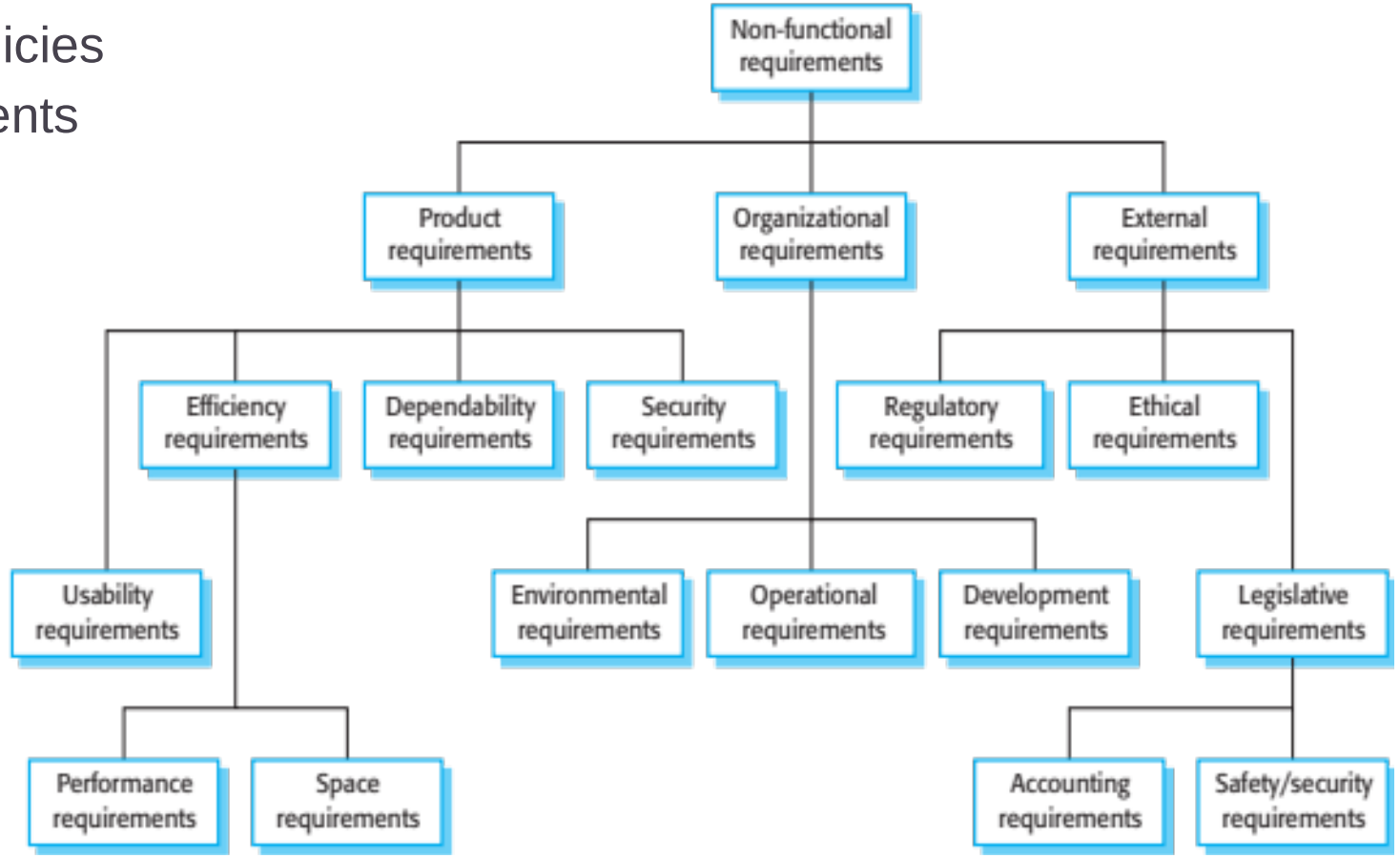
Requirements' dependency



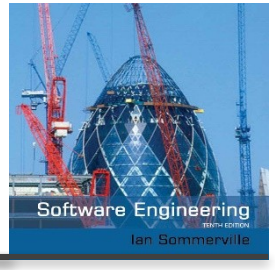
- **Requirements are not independent**.
- One requirement often constraints on another requirement .
- For example;
  - **Security is a non-functional requirement**
  - But **enforcing it will create a functional requirement of user authentication facility in the system.**

Non-functional requirements

- NFRs arise through:
  - Organizational policies
  - Product requirements
  - External sources

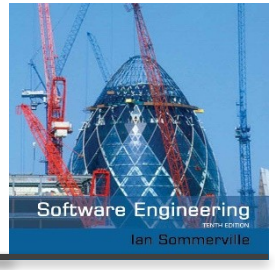


Non-functional requirements classification



- **Product Requirements:**
  - Specify constraint on runtime
- **Organizational Requirements:**
  - Derive from policies & procedures in customer’s and developer's organization
  - Specify the platform, programming languages to be used to develop system.
- **External Requirements:**
  - Requirements that are to be met necessarily to get the software approved by a regulatory body.
  - No ethical misuse / data leakage issue in the deployed product.

non-functional requirements for Hospital Management System



Product requirement

- The **system shall be available to all clinics during normal working hours** (Mon–Fri, 08:30–17:30).
- **Downtime** within normal working hours shall **not exceed five seconds in any one day.**

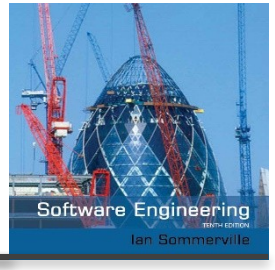
Organizational requirement

- **Users shall authenticate themselves using their ID cards**.
- (for this to enforce, every clinic needs a card reader facility which is difficult so alternate solution is to go for some other authentication means like SMS/email. So, one NFR may lead to another FR or NFR)

External requirement

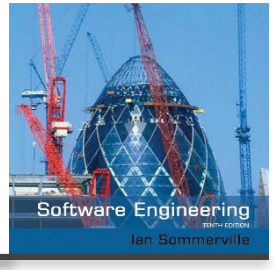
- **The system must respect patient privacy as stated in XXX-code of ethics**

Functional requirements for a Hospital Management System



- Functional requirements would be:
  - Sign up: Any employee/patient/doctor may register to the system
  - Sign In: After that he/she might be able to login.
  - User Id constraint: Each person using the system shall be uniquely identified by his or her 8-digit ID.
  - Search for doctors: A user shall be able to search the appointments lists of a doctor for all clinics.
  - Search for patients: The system shall generate a list of patients who are expected to attend appointments that day. (for doctors & receptionist)
  - Searching a doctor based on expertise: A patient can search a specialist wrt a domain
  - View data: Doctors can view patients history, diagnosis & medications.

Metrics for specifying nonfunctional requirements



Speed
- Transaction time,
- event response time

Size
- Memory usage

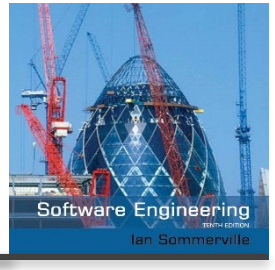
Ease of use
- Training time required for product usage

Reliability
- MTTF / system availability time

Robustness
- max. downtime in case of failure
- %age of failure occurrence
- Probability of data corruption after failure
- Transaction r back/not

Portability
- Diverse devices over which a system could run

Requirements imprecision

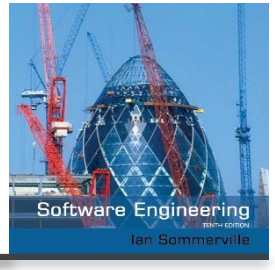


- **List every minute detail**.
- Less details might lead to ambiguous requirements.
- Ambiguous requirements lead to requirement misunderstanding and induce **system delay and increase costs**
- **Example:**

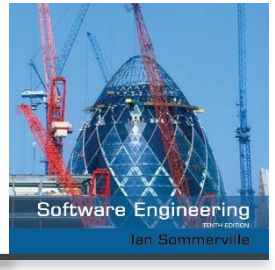
  "A user shall be able to **search** the appointments."

  - User intention – search a patient's appointments in all clinics;
  - Developer interpretation – User chooses clinic first & then search for a patient name in an individual clinic. (what if the patient wants to view the appointments in multiple clinics to observe a clash?)

Requirements completeness and consistency

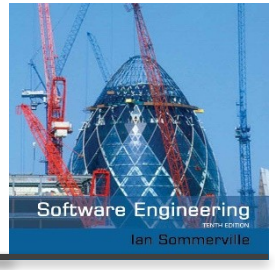


- In principle, **requirements should be both complete and consistent.**
- **Complete**
  - They should include descriptions of all facilities required.
- **Consistent**
  - There should be **no contradictions** in the descriptions of the system facilities.
- In practice, because of **system complexity**, it is **nearly impossible to produce a complete and consistent requirements document.**



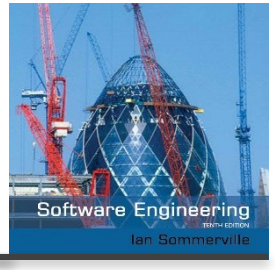
Requirements engineering processes

Requirements engineering processes

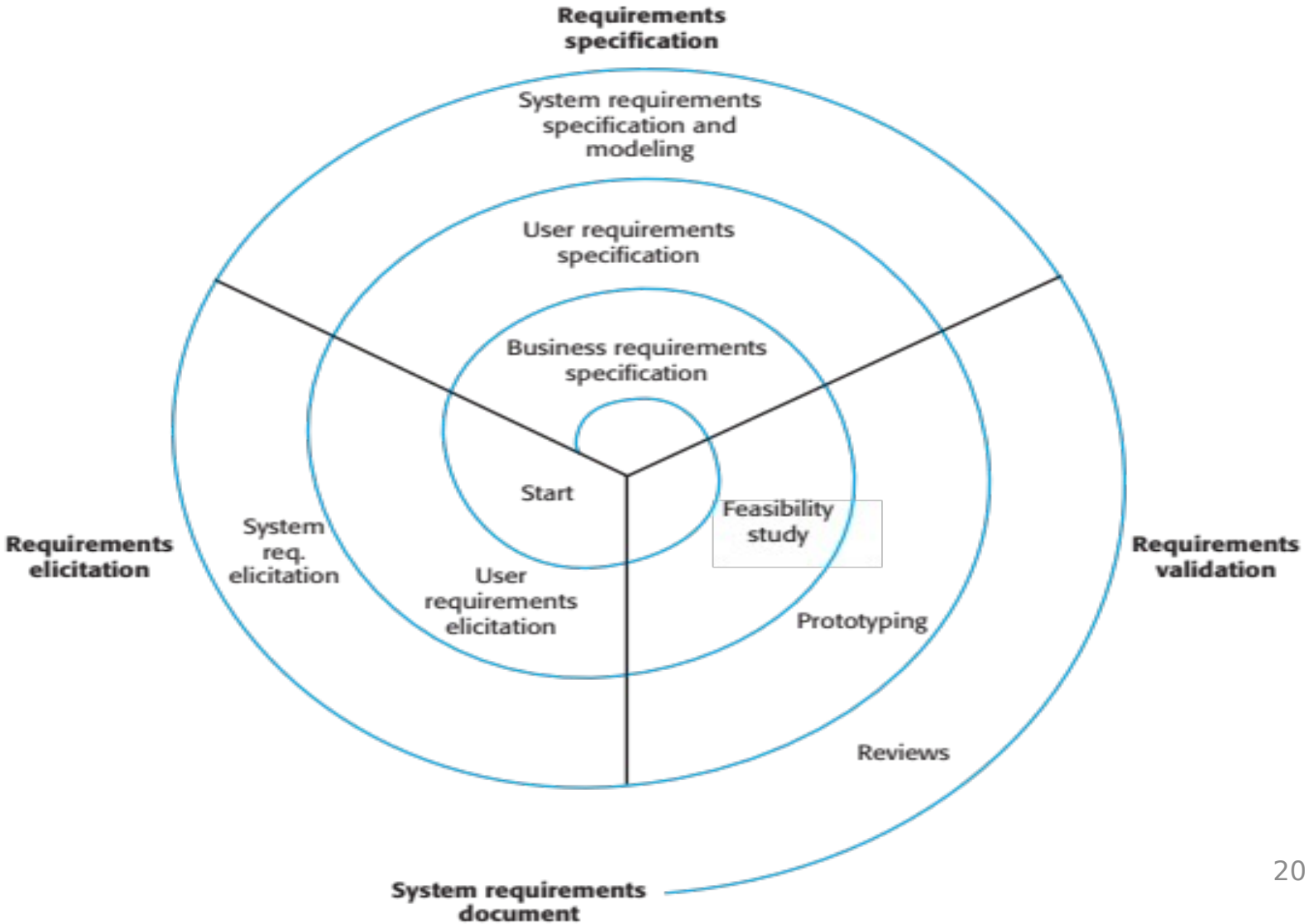


- **depends on:**
  - **the application domain**,
  - **the people** involved
  - and the **organisation developing the requirements.**
- Common generic activities are:
  - Requirements feasibility;
  - Requirements elicitation and analysis;
  - Requirements specification;
  - Requirements validation.
- In practice, **RE is an iterative activity in which these processes are interleaved.**

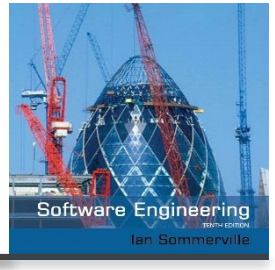
A spiral view of the requirements engineering process



- High level requirements view in inner rounds
- Complex requirements in outer ones
- Risk analysis
- Time and effort invested

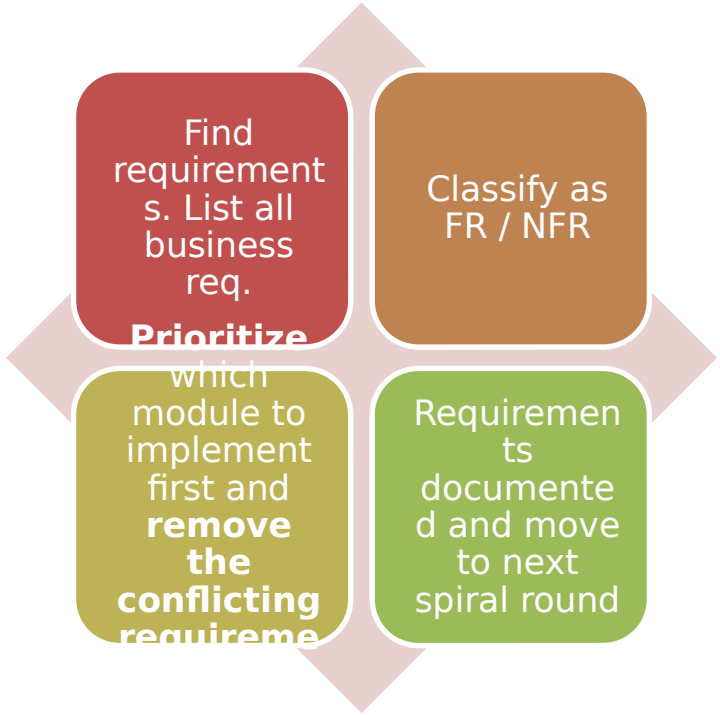


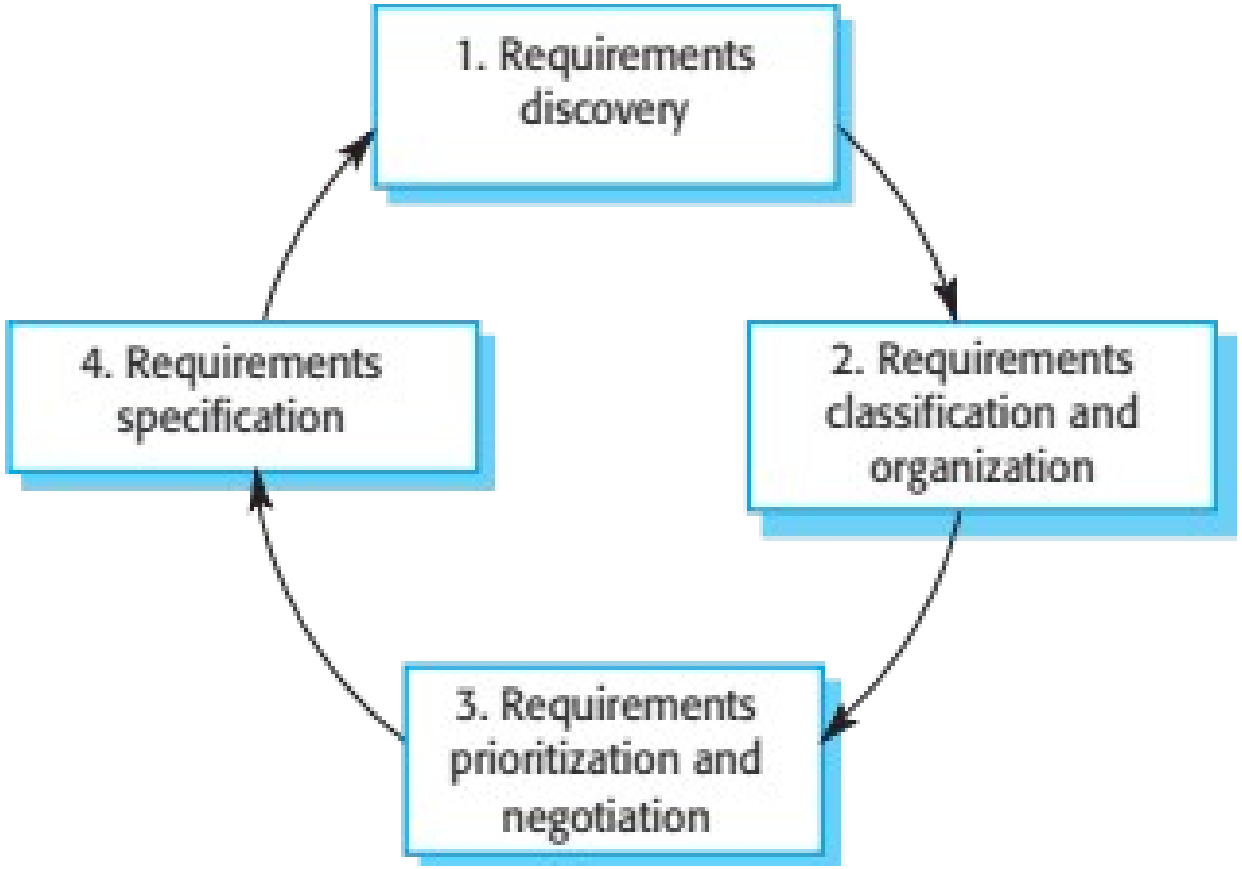
Requirements elicitation and analysis



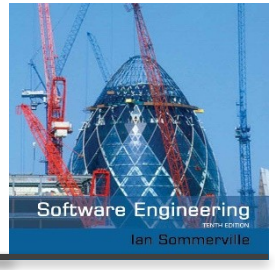
- technical staff works with customers to find out about:
  - the **type of application**,
  - the **services that the system should provide**
  - and the **system's hardware constraints**.
- May involve:
  - end-users,
  - managers,
  - engineers involved in maintenance,
  - domain experts, trade unions, etc.
  - These all are called as *stakeholders.*

Requirements elicitation and analysis process





Requirements discovery

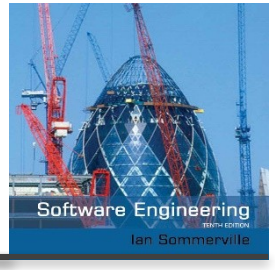


- gather information about the required and existing systems and extracting the user and system requirements from this information.
- Techniques used are:
  - Interviewing
  - Ethnography

Interviewing

- RE team conduct formal/informal interviews with stakeholders
- Types of interview
  - **Closed interviews:** pre-determined list of questions
  - **Open interviews:** no specific questionnaire, explore req. and refine them.
  - *Generally, a mixture of both types is used*
- Effective interviewing suggestions:
  - Be open-minded, **avoid pre-conceived ideas** about the requirements and are **willing to listen to stakeholders.**
  - You need to **prompt the user to talk about the system by suggesting requirements** rather than simply asking them what they want.

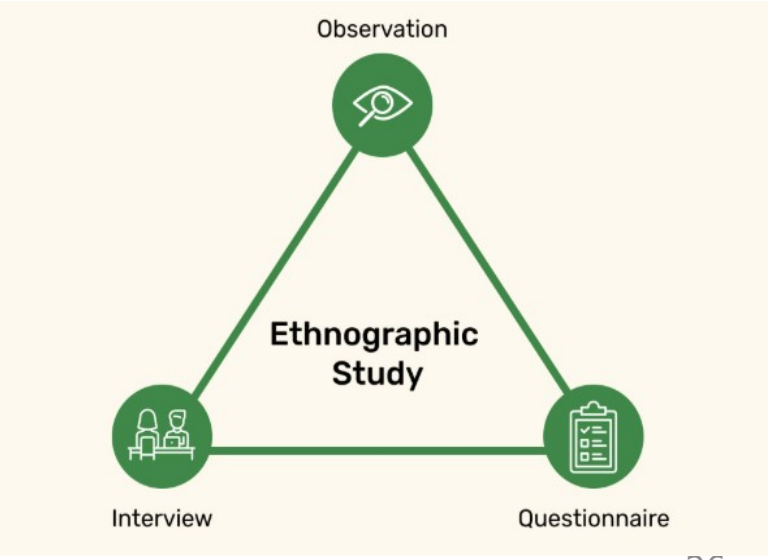
Problems with interviews



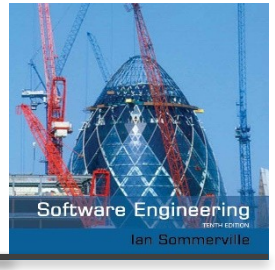
- **Communication gap** between the end user and RE team.
- **A prerequisite knowledge related to domain is needed**.
  - For example; Developing an e-commerce website must include transaction processing .

Ethnography

- A person spends time to observe and analyse how people actually work.
- People do not have to explain it to interviewer.
- Important Social and organisational factors are observed.
- Ethnographic studies have shown that work is usually richer and more complex than suggested by simple system models.

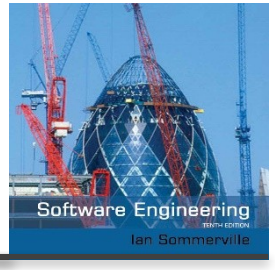


Scope of ethnography



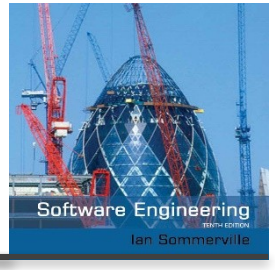
- Requirements are derived from the way in which people actually work rather than any pre-assumed ideal behaviour.
- **Based on user's activities, you may fine tune your system design**.
- **Good for understanding existing processes** but cannot suggest innovation to a system.

Focused ethnography

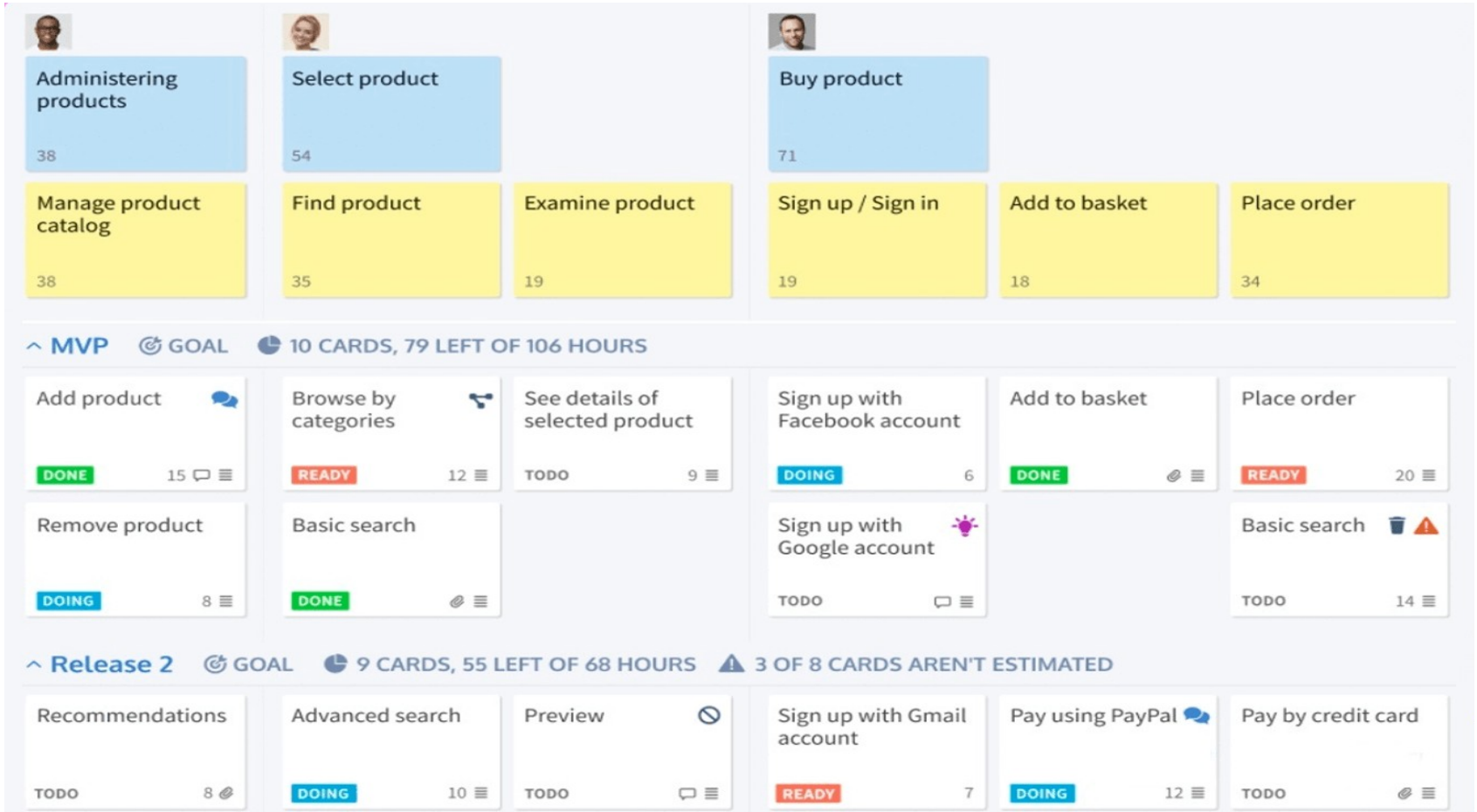


- Developed in a project studying the air traffic control process
- **Combines ethnography with prototyping**
- Prototype development results in unanswered questions which focus the ethnographic analysis.
- **ISSUE:**
  - it studies existing practices which may have some historical basis which is no longer relevant.

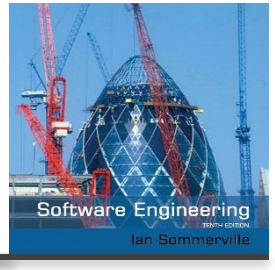
Stories and scenarios



- **Scenarios and user stories are real-life examples of how a system can be used for a task**.
- Since, it's a **practical situation, stakeholders can relate to them** and can comment on their situation with respect to the story.

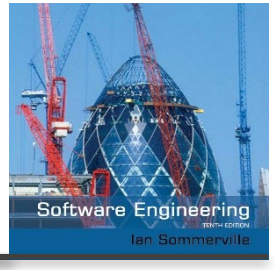


Scenarios



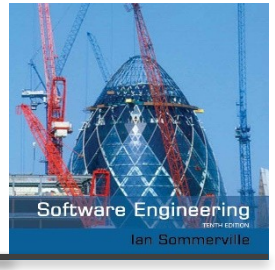
- A **structured form of user story**
- Scenarios should include
  - A description of the **starting situation**;
  - A description of the **normal flow of events**;
  - A description of **what can go wrong**;
  - **Information about other concurrent activities**;
  - **Scenario's finished state description**.

Problems of requirements elicitation



- Stakeholders don’t know what they really want.
- Stakeholders express requirements in their own terms.
- **conflicting requirements in case of different stake holders.**
  - Example: an HR , a stakeholder, explicitly requests to capture the age of an employee, but the Data Privacy team is saying that the age of the employee may not be captured or used in reporting.
- **Organisational constraints may influence the system requirements.**
- The **requirements change during the analysis process**.
  - New stakeholders may emerge and the business environment may change.

Health Club: Case Study

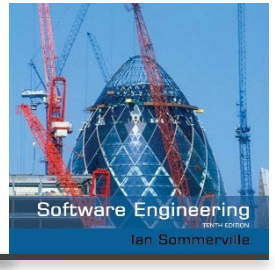


- The **XYZ Bank** is developing a **mobile banking application** to enable its customers to manage their finances securely and conveniently on mobile devices. Customers should be able to check balances, transfer funds, pay bills, apply for loans, and track transactions from their smartphones. Given that sensitive financial data is involved.
- **Actors:**
  - **Primary Actor**: Customer (End-User)
  - **Secondary Actors**:
    - **Bank's IT Team** (for back-end system architecture and server management)
    - **Security Team** (for securing user data and transactions)
    - **Business Analyst** (for gathering requirements)
    - **Mobile Developers** (for creating the app for both iOS and Android platforms)

NFR:

- Security
- Robust
- Usability
- portability
- performance
- Memory usage.

E-ZShop: Case Study



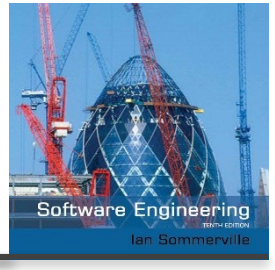
"E-ZShop" aims to revolutionize the online shopping experience by providing a user-friendly interface, seamless navigation, and secure transactions. The platform allows users to browse through a wide range of products, add items to their cart, make payments, and track their orders.

NFR:

- Performance
- Security
- Reliability
- usability

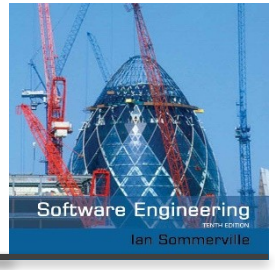
Requirements specification

Requirements specification



- **The process of writing down the user and system requirements in a requirements document.**
- The requirements would be: user/system requirements.
- **User req.:** in natural language
- **System req:** in natural language + graphical notations + mathematical system models
- Use **tables, forms , diagram to elaborate system working**

Ways of writing a system requirements specification



Natural language

- Written in sentences
- Every sentence is a requirement

Structured natural language

- Written using standard template
- Every sentence mentions details of requirements.

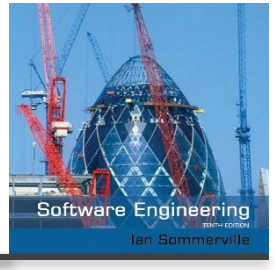
Graphical notations

- UML, use case and sequence diagrams

Mathematical specifications

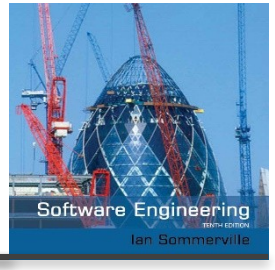
- Show via finite state machines

Requirements and design



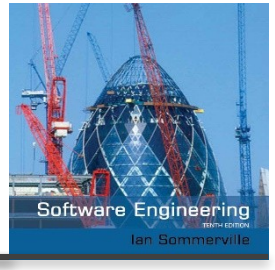
- requirements -> what the system should do
- Design -> describe how it does this.
- In practice, requirements and design are inseparable
  - A system architecture may be designed to structure the requirements;
  - Multiple subsystems in design may generate new requirements;
  - The use of a specific architecture to satisfy non-functional requirements may be a domain requirement.
    - An online banking application may be built using Python to minimize the calculation time and enforce reliability.

Guidelines for writing requirements



- Use a standard format for all requirements.
- Mention the mandatory & desirable requirements separately
- Highlight(Bold ,Italicize ,Underline) the key parts of the requirement.
- Avoid using technical words and abbreviations.
- Mention why a requirement is necessary.

Problems with natural language



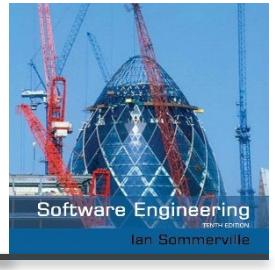
Ambiguous requirement details

Functional/non-functional Requirements not segregated

Requirements amalgamation will lead to longer deadlines in development

e.g., “when an employee enters the building, his entry is maintained in the log and his computer shall be booted”

Structured specifications

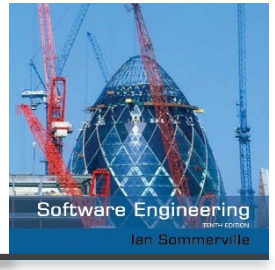


Writer is limited to follow the template.

Fit for defined requirement applications. E.g. Embedded systems

Not fit for systems having evolutionary requirements. E.g., Business applications

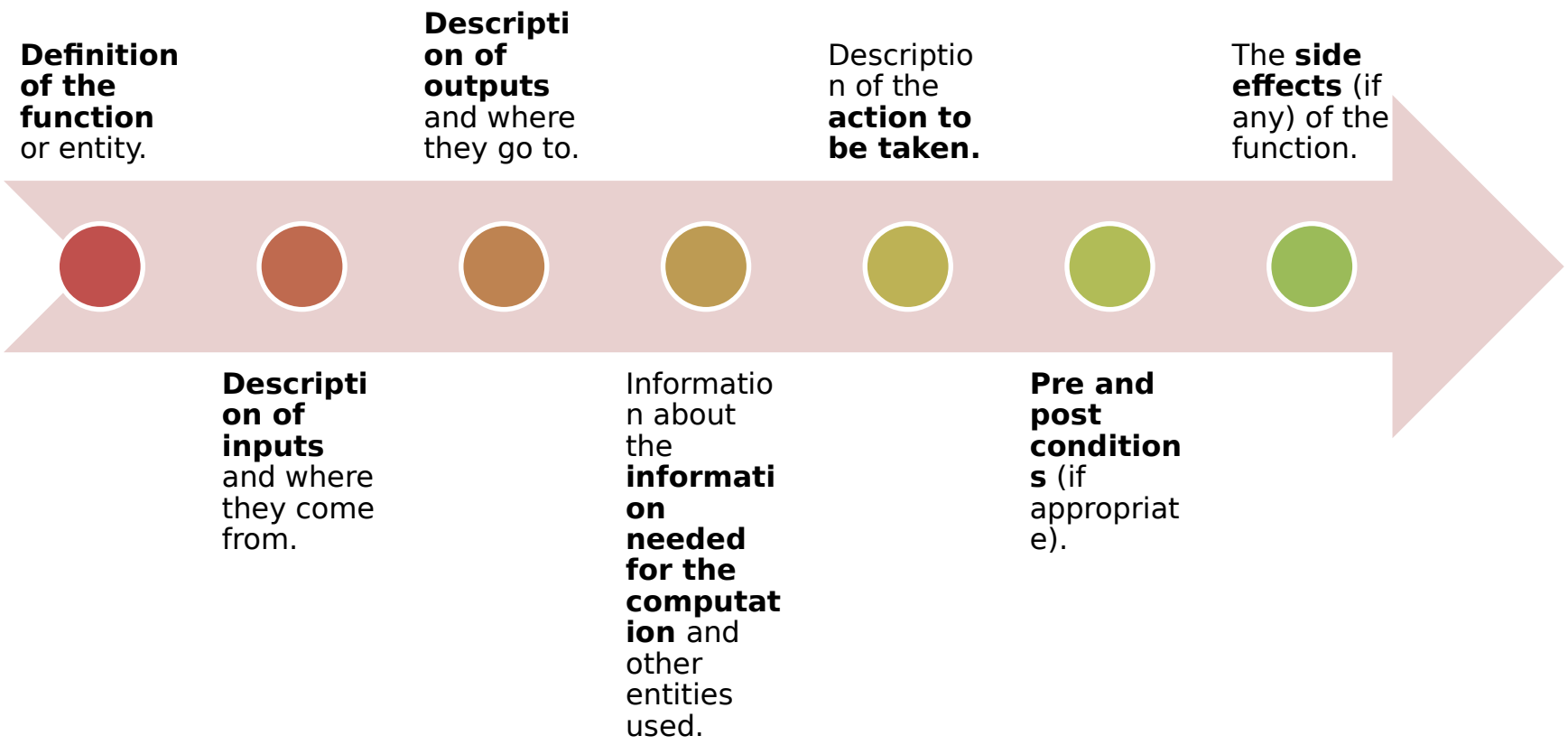
Example requirements for the login module



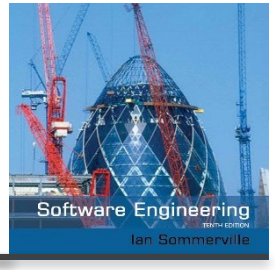
3.1 The system shall check the username and password and as per input login the system

3.2 The system shall close the session if it left unattended for more than 5 seconds.

Form-based specifications



Example requirements for the login module



Function authenticate user and login to system

Description

authenticate user and allow him to login if the username and password is correct. If the user enters the username/password wrong thrice, then the account needs to be recovered via phone no.

Inputs username, password

Source front-end form (mobile/standalone/web application)

Outputs login to system.

Action

Check the username and password entered by the user with the values in database. If match found, login. If not, then mark wrong input first time. If this happens thrice, then give account recovery option via passcode sent to phone number already saved in database. If correctly logged in session left unattended for 5 minutes, then logout the session.

Requirements

correct and valid username and password entered to login the system.

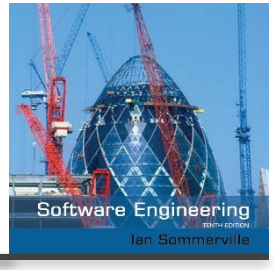
Pre-condition

must be a registered user

Post-condition

close the session if left unattended for >5 minutes

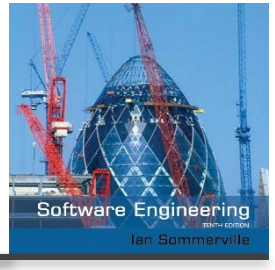
Tabular specification



- Used to supplement natural language.
- Particularly useful for multiple possible alternative and courses of action.

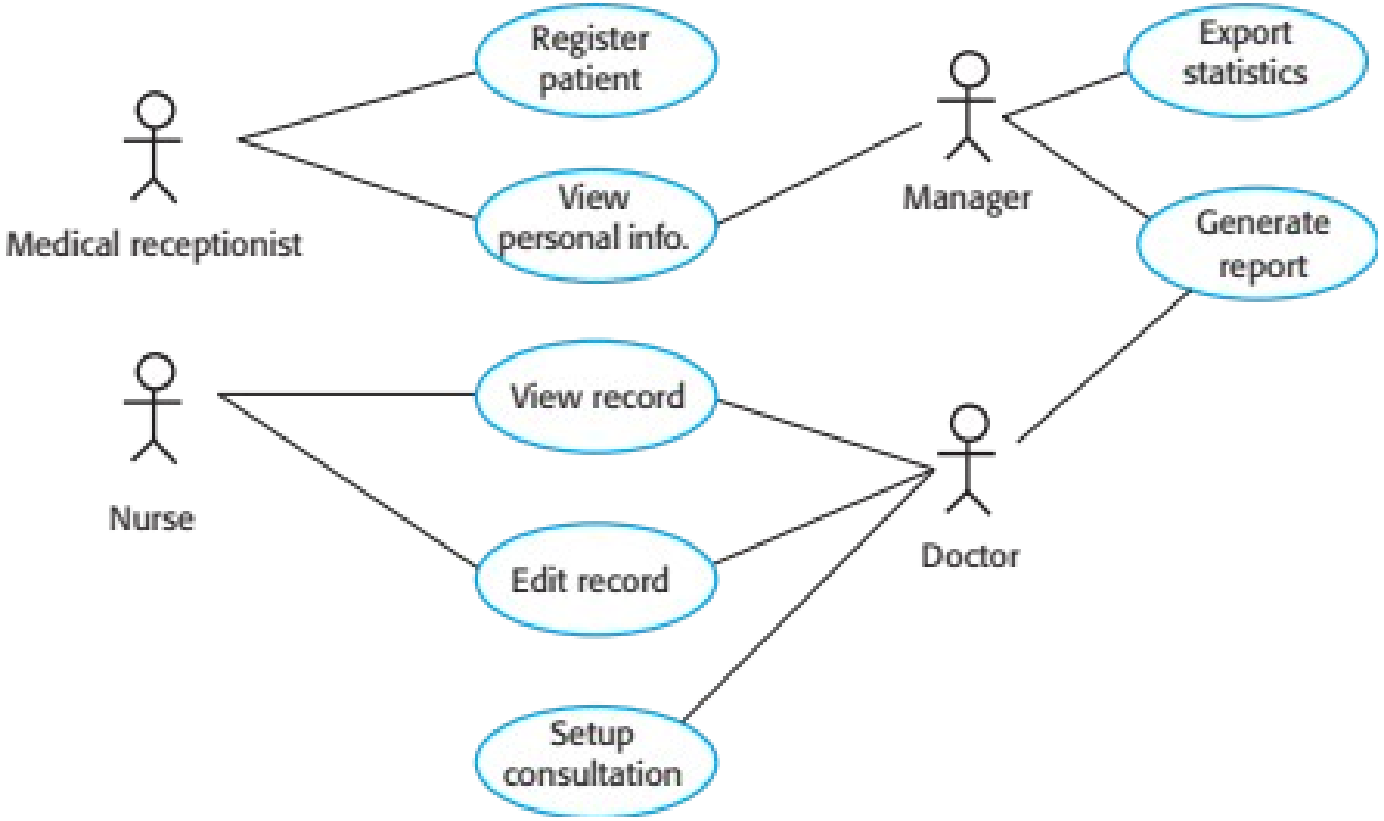
Condition	Action
Username correct , password wrong	Login fail
Username wrong, password wrong	Login fail
Username correct, password correct	Login

Use cases

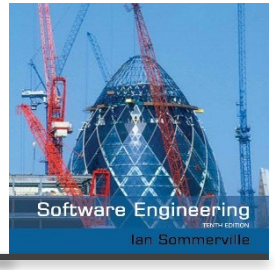


- **Use-cases are a kind of scenario that are included in the UML.**
- Use cases **identify the actors** in an interaction and which describe the interaction itself.
- A set of use cases should **describe all possible interactions with the system.**
- High-level graphical model.
- **UML diagrams may be added to use-cases to showcase the sequence of event processing in the system.**

Use cases for the Mentcare system

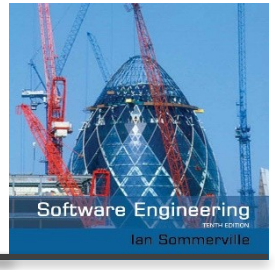


The software requirements document



- The software requirements document (SRD) is the **official statement of what is required of the system developers.**
- Should include:
  - definition of user requirements
  - a specification of the system requirements.
- It is **NOT a design document**.
- As far as possible, it should set of what the system should do rather than how it should do it.

Users of a requirements document



customer

- To have an idea of the requirements listed correctly

managers

- To plan a bid for the contract

System engineers

- To develop system

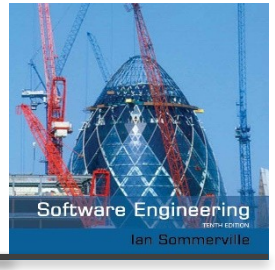
Test engineers

- To validate the system by writing and executing test cases

Maintenance engineers

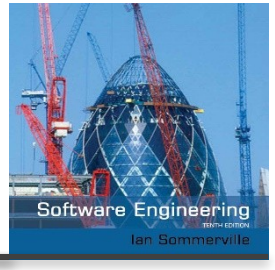
- For system evolution, check change possibility

The structure of a requirements document

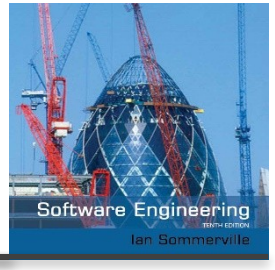


Chapter	Description
Preface	This should **define the expected readership of the document** and describe its **version history**, including a rationale for the creation of a new version and **a summary of the changes made in each version**.
Introduction	This should describe the **need for the system**. It should **briefly** describe **the system's functions** and explain how it will work with other systems. It should also describe how the **system fits into the overall business or strategic objectives of the organization commissioning the software**.
Glossary	This should **define the technical terms used in the document**. You should not make assumptions about the experience or expertise of the reader.
User requirements definition	Here, you **describe the services provided for the user**. The **nonfunctional system requirements** should also be described in this section. This d**escription may use natural language, diagrams, or other notations that are understandable to customers**. Product and process standards that must be followed should be specified.
System architecture	This chapter should present a high-level overview of the anticipated system architecture, showing the **distribution of functions across system modules**. **Reused architectural components should be highlighted**.

The structure of a requirements document

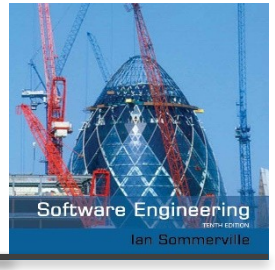


Chapter	Description
System requirements specification	This should **describe the functional and nonfunctional requirements in more detail**. If necessary, further detail may also be added to the nonfunctional requirements. Interfaces to other systems may be defined.
System models	This might **include graphical system models showing the relationships between the system components and the system and its environment**. Examples of possible models are object models, data-flow models, or semantic data models.
System evolution	This should describe the fundamental assumptions on which the system is based, and **any anticipated changes due to hardware evolution, changing user needs, and so on**. This section is useful for system designers as it may help them avoid design decisions that would constrain likely future changes to the system.
Appendices	These should provide detailed, specific information that is related to the application being developed; for example, **hardware and database descriptions**. Hardware requirements define the minimal and optimal configurations for the system. Database requirements define the logical organization of the data used by the system and the relationships between data.
Index	Several indexes to the document may be included. As well as a normal alphabetic index, there may be an **index of diagrams, an index of functions**, and so on.



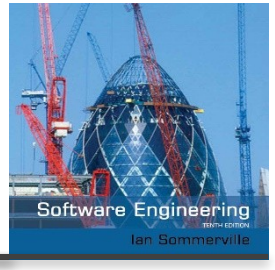
Requirements validation

Requirements validation



- Process of checking that **requirements are defining the system as per the customer desires.**
- **Overlapped with elicitation and analysis phase**
- **Since rework costs are very high so it's one of the most critical steps**
  - Fixing a requirements error after delivery may cost up to 100 times the cost of fixing an implementation error.
  - As change in requirement leads to change in design, implementation and testing as well.

Types of checks done to validate requirements



Validity.

- Either the **requirement reflecting real needs of user**?
- As user requirements might have changed after elicitation.

Consistency.

- **No contradictory constraints** or requirements
- **No different description of a same system**

Completeness.

- **Requirements listed must include all system's functionality and constraints**

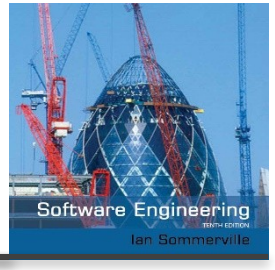
Realism.

- Either a req. could be implemented via defined technology.
- Could it be done in available budget, and time?

Verifiability.

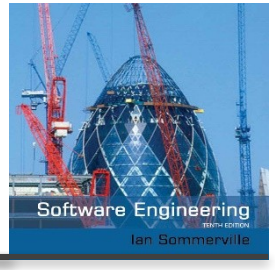
- Can the requirements be checked?

Requirements validation techniques

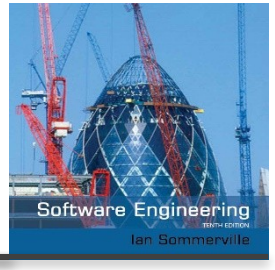


- Requirements review
  - procedure done by customer and developers
  - read whole requirement specification document.
  - Check for error, anomalies, inconsistencies
  - If detected
  - Record it
  - then both parties negotiate it to rectify it
- Prototyping
- Test-case generation/test-driven development
  - Involves input, output and basic functional code
  - Check if test cases satisfies requirements.

Requirements validation challenges



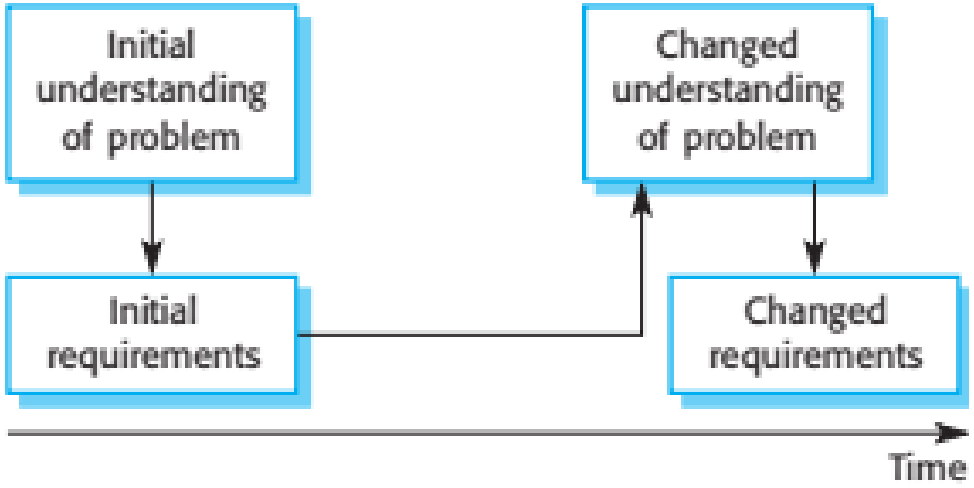
- Its really difficult for developers & users to do the abstract analysis and find all the loop holes.
- So, it is possible that the requirements may change after validation.



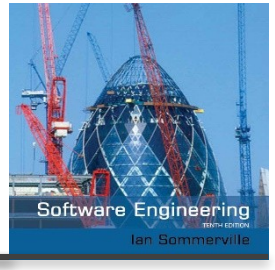
Requirements change

Requirements evolution

- System errors
- Omission in original requirements
- Emerging business requirements

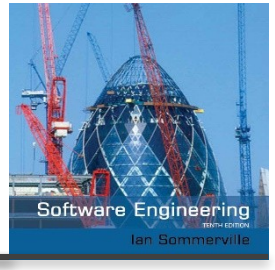


Changing requirements ~ reason 1



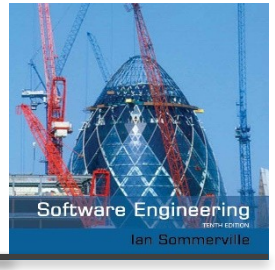
- The business and technical environment of the system always changes after installation.
  - **New hardware** may be introduced,
  - **business priorities may change** (with consequent changes in the system support required)
  - Example: Installing RFID attendance system and integrate it to employees portal.

Changing requirements ~ reason 2



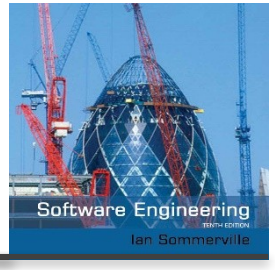
- The people who pay for a system and the users of that system are rarely the same people.
  - **System customers impose requirements because of** organizational and **budgetary constraints**.
  - These may **conflict with end-user requirements** and, **after delivery, new features may have to be added for user support** if the system is to meet its goals.
  - Example: at first, an employee management system only contains limited necessary functionality like attendance review, leave balance and all, now users want to add their APR forms and PF statements as well.

Changing requirements ~ reason 3



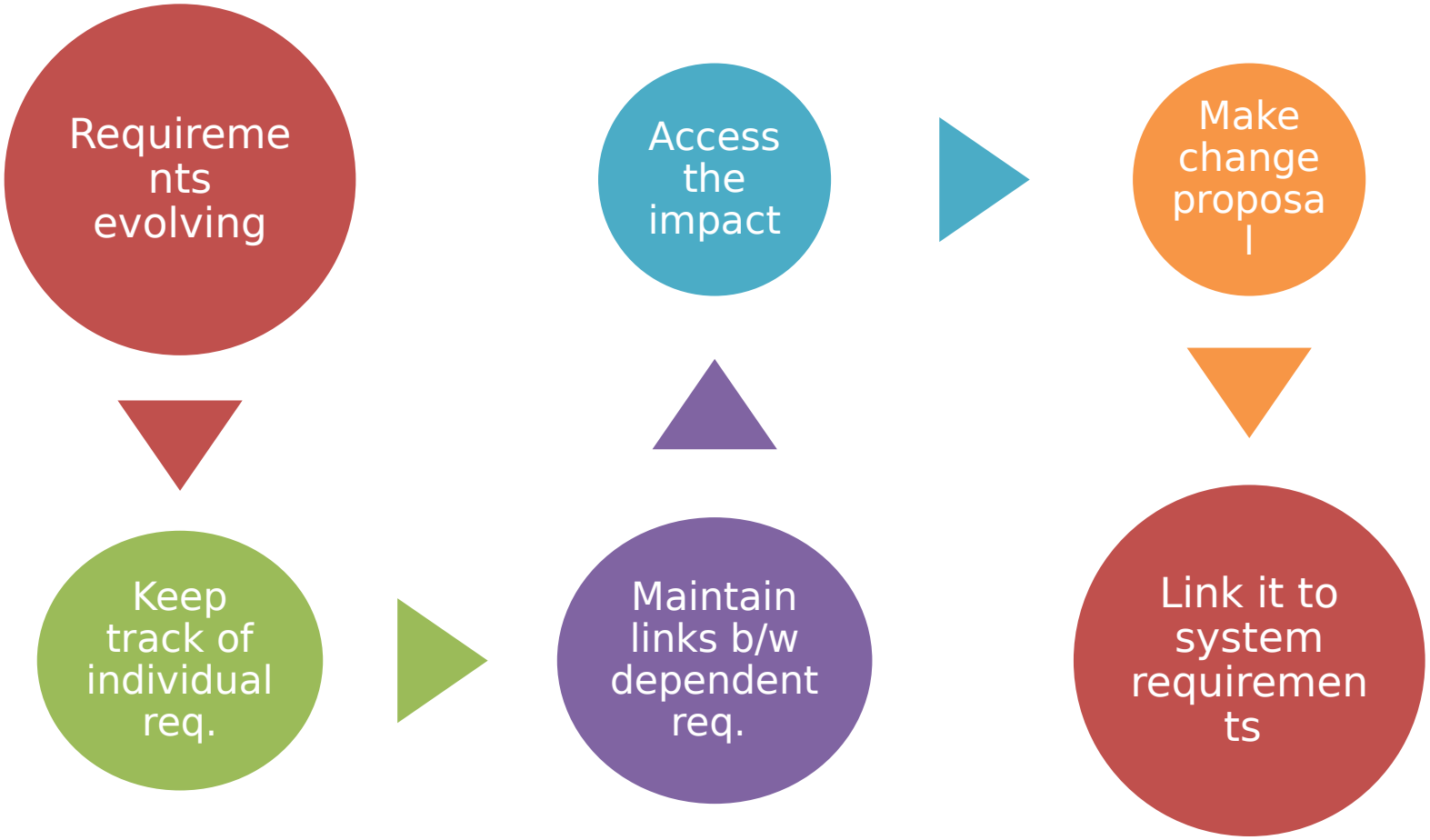
- **Large systems** usually have a **diverse user community**, with many users having **different requirements and priorities** that may be **conflicting** or contradictory.
  - The final system **requirements are inevitably a compromise between them**
  - with experience, it is often discovered that the **balance of support given to different users has to be changed depending on priority**.

Requirements management

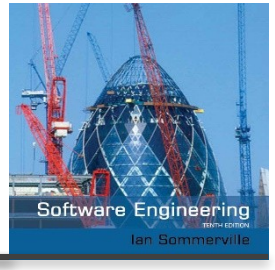


- It is a process of **managing changing requirements during the requirements engineering process and system development.**
- New requirements may emerge
  - as a system is being developed and
  - after it has gone into use.

Requirements management

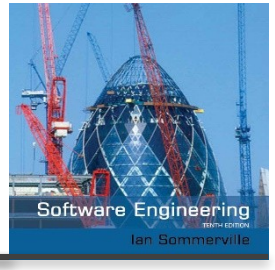


Requirements management planning



- ***Requirements identification***
  - Each requirement must be uniquely identified so that it can be cross-referenced with other requirements. Requirement Ids used for each requirement.
- ***A change management process***
  - This is the set of activities that **assess the impact and cost of changes.**
- ***Traceability policies***
  - These policies **define the relationships between each requirement** and **between the requirements and the system design that should be recorded**.
- ***Tool support***
  - Tools support that is offered from some requirements management systems to spreadsheets and simple database systems.

Requirements change management



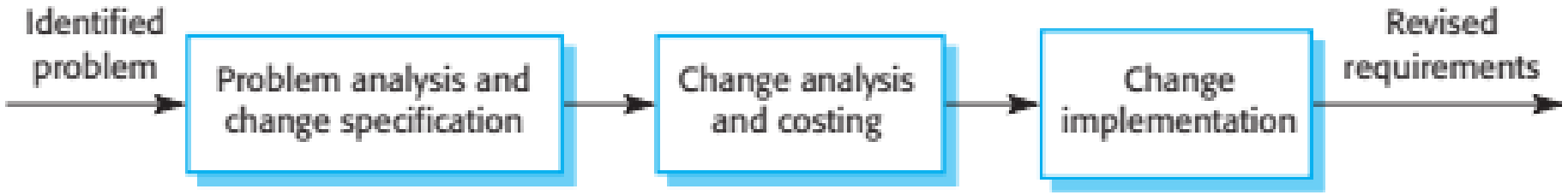
Problem analysis and change specification

- check validity of change proposal
- pass on to change requestor
- who will either suggest a better specific requirements change proposal or

Change analysis and costing

- The effect of the proposed change is assessed using traceability information and general knowledge of the system requirements.
- After analysis, proceed/not to proceed with the requirements change is

Change implementation

- The document should be organized so that changes can be easily implemented. Like modular documentation should be done to cater changes.

Requirements change management